

KAFKA

1. WHAT PROBLEM KAFKA SOLVES

In traditional systems, applications directly interact with multiple downstream systems:

Order Service → Database
 → Analytics
 → Notifications

Problems:

- Tight coupling between systems
- If one system fails, it impacts others
- No buffering mechanism
- No replay capability
- Difficult to scale
- Mostly batch-based processing

Event-Based Architecture

Instead of directly calling systems, applications generate **events**.

Example:

Order Created → Event

These events are sent to a central system.

Kafka's Role

Kafka acts as:

- A **central event streaming platform**
- A **buffer between systems**
- A **durable storage system**
- A **real-time data pipeline backbone**

Flow:

Producer → Kafka → Multiple Consumers

2. KAFKA CORE CONCEPTS

Topic

A **topic** is a logical stream of data.

Examples:

orders
payments
clickstream

It is similar to:

- a table (in databases)
- a folder (in storage systems)

Partition

Each topic is divided into **partitions**.

orders topic:

Partition 0

Partition 1

Partition 2

Why partitions exist:

- Enable parallel processing
- Enable horizontal scaling
- Improve throughput

Partition Internals

Each partition is an **append-only log**:

Partition 0:

0 → order1

1 → order2

2 → order3

Offset

- Offset is the **position of a message inside a partition**
- It is unique within a partition

Important:

- Offsets are **not global**
- Each partition maintains its own offsets

Producer

A **producer** sends data to Kafka.

Examples:

- Order service
- Payment service
- Web application

Responsibilities:

- Choose topic
- Decide partition
- Send data

Consumer

A **consumer** reads data from Kafka.

Examples:

- Spark job
- Analytics system
- Notification service

Important behavior:

- Kafka is **pull-based**
- Consumers decide:
 - when to read
 - how much to read

Consumer Group

A **consumer group** is a set of consumers working together.

Example:

Group G1:

C1 C2 C3

Kafka distributes partitions among consumers.

Key Rule

- One partition is consumed by only **one consumer in a group**

Example

Partitions: P0 P1 P2

Consumers: C1 C2

Assignment:

C1 → P0, P1

C2 → P2

Ordering Guarantee

- Ordering is guaranteed **only within a partition**
- Not across partitions

3. KAFKA ARCHITECTURE

Components

- **Broker** → Kafka server

- **Cluster** → group of brokers
- **Topic** → **Partitions** → **Logs**

Data Flow

Producer → Broker (Leader) → Followers → Consumer

Leader & Followers

Each partition has:

- 1 Leader → handles reads/writes
- Multiple Followers → replicate data

ISR (In-Sync Replicas)

- Set of replicas that are up-to-date
- Used for fault tolerance

4. PRODUCER INTERNALS

Partition Selection

If key present:

partition = hash(key) % partitions

Else:

round-robin

Batching

Producer does not send one message at a time.

It batches messages:

- `batch.size`
- `linger.ms`

Benefits:

- High throughput
- Efficient network usage

Acknowledgments (acks)

Value	Meaning
0	No guarantee
1	Leader only
all	Leader + replicas

Retries

Producer retries sending messages on failure.

Idempotence

`enable.idempotence=true`

Prevents duplicate writes.

5. CONSUMER INTERNALS

Consumer Workflow

1. Connect to broker
2. Join consumer group
3. Get partition assignment
4. Read messages
5. Commit offsets

Offset Management

Offsets are stored in Kafka internally.

Auto Commit

- Offsets committed automatically
- Risk: data loss

Manual Commit

- Controlled commit
- Safer but can cause duplicates

6. CONSUMER GROUP REBALANCING

What triggers rebalance?

- New consumer joins
- Consumer leaves
- Topic partition changes

What happens during rebalance?

1. Stop consumption
2. Reassign partitions
3. Resume

Impact

- Temporary downtime
- Possible duplicates

7. STORAGE INTERNALS

Log Structure

Kafka stores data in:

.log files
.index files
.timeindex files

Segment Files

Data is split into segments:

`segment.bytes = 1GB`

Why Kafka is Fast

- Sequential disk writes
- OS page cache
- Zero-copy transfer
- Batching

8. RETENTION

Kafka does NOT delete data after consumption.

Retention is based on:

- Time (`retention.ms`)
- Size (`retention.bytes`)

Example

`retention.ms = 1 day`

Data older than 1 day is deleted.

9. DELIVERY GUARANTEES

Type	Description
At most once	possible data loss
At least once	possible duplicates
Exactly once	no loss, no duplicates

Practical Usage

- Most systems use **at least once**
- Exactly once requires:
 - idempotence
 - transactions

10. FAULT TOLERANCE

Replication

Each partition is replicated across brokers.

Leader Failure

- New leader is elected from ISR
- No data loss if ISR is healthy

Configurations

- replication.factor
- min.insync.replicas

PERFORMANCE TUNING

1. PRODUCER TUNING (WRITE PATH OPTIMIZATION)

1.1 batch.size

What it is:

- Size of batch (in bytes) before sending to broker

Default:

16384 bytes (16 KB)

How it works:

Instead of:

Send message1
Send message2
Send message3

Kafka does:

Batch → [msg1, msg2, msg3] → send together

Impact:

Small batch	Large batch
Low latency	High throughput
More network calls	Fewer network calls

When to increase:

- High throughput pipelines
- Streaming logs
- Bulk ingestion

Risk:

- Increased latency

1.2 linger.ms

What it is:

- Time producer waits before sending batch

Example:

`linger.ms = 0` → send immediately

`linger.ms = 100` → wait 100ms to accumulate batch

Trade-off:

Low linger	High linger
Low latency	Better batching
Lower throughput	Higher throughput

Real-world use:

- Real-time alerts → low linger
- Data pipelines → higher linger

1.3 compression.type

Options:

none
gzip
snappy
lz4
zstd

Why compression matters:

- Reduces network usage
- Improves throughput

Trade-off:

Compression	CPU	Network
none	low	high
gzip	high	low
snappy	medium	good balance

1.4 acks (VERY IMPORTANT)

Value	Behavior
0	fastest, no guarantee
1	leader only
all	safest

Production standard:

acks=all

1.5 retries

- Retry failed sends
- Prevent transient failures

1.6 enable.idempotence

true → no duplicates from producer

PRODUCER SUMMARY

- Throughput = batch.size + linger.ms

- Reliability = acks + retries + idempotence

2. BROKER TUNING (CLUSTER LEVEL)

2.1 partitions

Most important scaling parameter

Rule:

More partitions → more parallelism

But:

Too many partitions →

- Metadata overhead
- Memory usage
- Rebalancing cost

Practical guideline:

- Start small (2–6)
- Scale based on throughput

2.2 replication.factor

1 → no fault tolerance
3 → production standard

Trade-off:

Low replication	High replication
fast	durable
risky	safe

2.3 min.insync.replicas

Works with:

`acks=all`

Example:

`replication.factor=3`
`min.insync.replicas=2`

At least 2 brokers must confirm write

2.4 retention

`retention.ms`
`retention.bytes`

Important understanding:

Kafka = log system
NOT queue

2.5 segment.bytes

- Size of log segment

Smaller segments:

- Faster cleanup
- More files

3. CONSUMER TUNING (READ PATH)

3.1 fetch.min.bytes

Minimum data to fetch

Behavior:

- Wait until enough data accumulates

3.2 max.poll.records

Max messages per poll

Impact:

- Controls processing batch size

3.3 parallelism

Controlled by:

number of partitions

3.4 auto.offset.reset

earliest → read from beginning

latest → read new data only

3.5 enable.auto.commit

true → automatic

false → manual control

CONSUMER SUMMARY

- Performance = fetch + poll tuning
- Reliability = offset control

COMPLETE KAFKA LAB

LAB GOAL

You will demonstrate:

Producer → Kafka → Consumer (LIVE)
↓
Spark (LIVE)

Students will **see real-time streaming**

STEP 0 — START ENVIRONMENT

Run:

```
docker-compose up -d
```

What this does:

- Starts Kafka
- Starts Zookeeper
- Starts Spark
- Starts Jupyter

(From your setup)

Verify:

```
docker ps
```

You should see:

- kafka
- zookeeper
- spark
- jupyter

STEP 1 — ENTER KAFKA CONTAINER

```
docker exec -it kafka bash
```

Why?

Kafka CLI tools exist **inside this container**

STEP 2 — CREATE TOPIC

```
kafka-topics \  
--create \  
--topic orders \  
--bootstrap-server localhost:9092 \  
--partitions 2 \  
--replication-factor 1
```

Explanation (IMPORTANT)

kafka-topics

Kafka tool to manage topics

--create

Create new topic

--topic orders

Name of topic → "orders"

--bootstrap-server localhost:9092

Address of Kafka broker

--partitions 2

Topic will have 2 partitions

Meaning:

- Data will be split into 2 parts
- Allows parallel processing

--replication-factor 1

Only 1 copy of data

Since local setup → no replication

STEP 3 — VERIFY TOPIC

```
kafka-topics --list --bootstrap-server localhost:9092
```

```
kafka-topics --describe \  
--topic orders \  
--bootstrap-server localhost:9092
```

What to explain

Output shows:

- Partition count
- Leader broker
- Replicas

STEP 4 — START CONSUMER (FIRST)

Open **Terminal 1**

```
kafka-console-consumer \  
--topic orders \  
--bootstrap-server localhost:9092
```

Explanation

kafka-console-consumer

Tool to read messages

--topic orders

Read from "orders"

--bootstrap-server

Connect to Kafka

IMPORTANT

Nothing prints yet → waiting for data
This is a **live listener**

STEP 5 — START PRODUCER

Open **Terminal 2**

```
kafka-console-producer \  
--topic orders \  
--bootstrap-server localhost:9092
```

Explanation

kafka-console-producer

Tool to send messages

--topic orders

Send data to "orders"

--bootstrap-server

Connect to Kafka

STEP 6 — SEND LIVE DATA

Type in producer:

```
order1,100  
order2,200  
order3,300
```

WHAT HAPPENS

Immediately appears in consumer

- Data is sent instantly
- Kafka stores it
- Consumer reads instantly

This is **real-time streaming**

STEP 7 — CONSUMER GROUP

Stop previous consumer

Start this:

```
kafka-console-consumer \  
--topic orders \  
--group g1 \  
--bootstrap-server localhost:9092
```

Explanation

--group g1

- Creates consumer group named "g1"

- Kafka tracks progress (offsets)

Behavior

- Kafka remembers what this group read
- If restarted → resumes from last position

STEP 8 — CHECK OFFSETS

```
kafka-consumer-groups \  
--describe \  
--group g1 \  
--bootstrap-server localhost:9092
```

Explain output

Column	Meaning
CURRENT-OFFSET	last read message
LOG-END-OFFSET	latest message
LAG	messages not yet read

STEP 9 — START SPARK STREAM (LIVE PIPELINE)

Open Jupyter

<http://localhost:8888>

Password:

training123

Run this code:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("KafkaLab") \
    .getOrCreate()

df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "orders") \
    .load()

df.selectExpr("CAST(value AS STRING)") \
    .writeStream \
    .format("console") \
    .start() \
    .awaitTermination()
```

EXPLANATION

readStream

- Enables streaming mode
- Not batch

.format("kafka")

- Source is Kafka

kafka.bootstrap.servers = kafka:9092

- Kafka container inside Docker network

subscribe = orders

- Read from "orders" topic

.load()

- Start reading stream

CAST(value AS STRING)

- Kafka stores data as bytes
- Convert to readable text

.writeStream

- Continuous output

`.format("console")`

- Print output

`.start()`

- Start streaming job

`.awaitTermination()`

- Keep job running

STEP 10 — FINAL LIVE TEST

Go back to producer and send:

order10,500

order11,700

WHAT HAPPENS

- Consumer prints instantly
- Spark prints instantly

FINAL FLOW

Producer → Kafka → Consumer

↓

Spark

